Four Requests, Traced End to End

Part 4 of 5: following real features through every layer

LOGIN · CREATE A STUDENT · LOG A LESSON · GIVE A MARK

If you can talk through these four, you can talk through the whole codebase — every other endpoint is a variation on one of them.

Walkthrough 1 — Logging in

```
POST /api/auth/login
{ "email": "anna@school.example", "password": "..." }
```

Step 1: the endpoint

In `AuthEndpoints.cs`, on a group that is deliberately *not* under `/api`'s blanket `.RequireAuthorization()`:

```
group.MapPost("/login", async (LoginUserDto dto, IUserService users, CancellationToken ct) =>
    Results.Ok(await users.LoginAsync(dto, ct)));
```

Before the handler body runs, `ValidationEndpointFilter` has already checked that the email looks like an email and the password is not empty.

Step 2: verify

Look up by email, verify the hash, check deactivation — in that order, for the reasons in Part 3.

Step 3: resolve the profile

Here is the subtle part. The `User` row holds the login and the role, but not which school the person belongs to. That lives on their profile row:

```
case UserRole.Teacher:
{
    var teacher = await teacherRepository.GetByUserIdAsync(user.Id, ct);
    return (teacher?.SchoolId, teacher?.Id, null);
}
```

And `GetByUserIdAsync` uses `.IgnoreQueryFilters()`. Ask yourself why, because it is a good interview question: the query filter compares `SchoolId` against the current user's school claim — but we are in the middle of *issuing* that claim. It does not exist yet, so the filter would compare against `NULL`, match nothing, and every login would come back school-less. The filter must be bypassed precisely because this query is what determines the school.

Step 4: mint the token

Claims are assembled, signed with HMAC-SHA256, and returned along with the user's name, role and ids so the front end can render without a second call.

```
POST /api/auth/login → ValidationEndpointFilter — shape ok? → UserService.LoginAsync →
GetByEmailAsync — tracked, in case a rehash is needed → VerifyHashedPassword → DeactivatedAt check →
ResolveProfileAsync — IgnoreQueryFilters: no school claim yet → build + sign JWT ← 200
AuthResponseDto
```

Walkthrough 2 — An admin creates a student

```
POST /api/students
{ "firstName": "Ben", "lastName": "Novak", "email": "ben@school.example",
  "password": "...", "dateOfBirth": "2010-04-02", "personalNumber": "1002004500006" }
```

This one demonstrates a **multi-entity write in a single transaction**. A student is really two rows: a `User` (the login) and a `Student` (the profile).

The service

```
public async Task<StudentDetailDto> CreateAsync(StudentCreateDto dto, CancellationToken ct)
{
    if (await students.PersonalNumberExistsAsync(dto.PersonalNumber, null, ct))
        throw new ConflictException("This personal number is already registered.");

    var user = await users.BuildLoginAsync(
        dto.FirstName, dto.LastName, dto.Email, dto.Password, UserRole.Student, ct);

    var student = mapper.Map<Student>(dto);
    student.User = user;
    student.SchoolId = currentUser.RequireSchoolId();

    await students.AddAsync(student);
    await students.SaveChangesAsync(ct);

    return await GetByIdAsync(student.Id, ct);
}
```

Six things are worth pointing at:

1. **The uniqueness check crosses schools.** A national ID is unique nationally, so the same child cannot be enrolled at two schools under two records.
2. **The error message does not echo the value back.** Returning *"personal number 1002004500006 already exists"* would confirm to School A that a specific child is enrolled at School B — an existence oracle over national IDs.
3. `BuildLoginAsync` **builds but does not save.** It adds the `User` to the change tracker and returns it. One place owns email-uniqueness and hashing for all three profile types.
4. `student.User = user` uses the navigation property, so EF works out the foreign key and the insert order itself.
5. `SchoolId` **comes from the token**, never the request body. This is the line that makes the tenancy boundary real, and it recurs in every create.
6. **One** `SaveChangesAsync` **for both rows.** EF wraps it in a single transaction, so you cannot end up with a login and no student.

On the way out, `SchoolScopeInterceptor` confirms the new `Student`'s school matches the caller's, and `AuditSaveChangesInterceptor` adds `Added User (...)` and `Added Student (...)` rows to the same

transaction.

Where AutoMapper fits

```
private IMappingExpression<TSource, TDestination> CreateWriteMap<TSource, TDestination>()
```

`mapper.Map<Student>(dto)` copies `DateOfBirth` and `PersonalNumber` across by name. It does *not* touch `Id`, `UserId`, `SchoolId` or the navigations, because `CreateWriteMap` explicitly ignores every destination property the source does not name. Those stay the service's decision, which is exactly where a tenancy field belongs.

Walkthrough 3 — A teacher logs a lesson

```
POST /api/diary-entries
{ "classId": "...", "subjectId": "...", "teacherId": "...",
  "date": "2026-03-04", "period": 3, "topic": "Quadratic equations" }
```

The most interesting write in the system: it enforces four separate rules and creates a whole register as a side effect.

The checks, in order

```
if (currentUser.IsTeacher && dto.TeacherId != currentUser.TeacherId)
    throw new ForbiddenException("You can only log lessons that you taught yourself.");

await access.EnsureCanTeachClassAsync(dto.ClassId, ct);

if (!await teacherSubjects.ExistsAsync(dto.TeacherId, dto.SubjectId, ct))
    throw new BusinessException("This teacher is not qualified to teach that subject.");

var everAssigned = (await assignments.GetByClassAsync(dto.ClassId, true, ct))
    .Any(a => a.SubjectId == dto.SubjectId);

if (!everAssigned)
    throw new BusinessException("That subject is not taught in this class.");

if (await diaryEntries.SlotTakenAsync(dto.ClassId, dto.Date, dto.Period, ct))
    throw new ConflictException("A lesson is already recorded for this class in that period.");
```

CHECK	PREVENTS
Teacher is themselves	Logging a lesson under a colleague's name. Admins may, on someone's behalf.
`EnsureCanTeachClassAsync`	Opening a register for a class you have nothing to do with.
Qualified for the subject	A history teacher filing a maths lesson.
Subject taught in this class	Filing a chemistry lesson for a class that has no chemistry.
Slot free	Two lessons in the same class, same day, same period.

Note check 3 tests *qualification* (`TeacherSubject`), not *assignment* (`ClassSubjectTeacher`). That is deliberate: a qualified substitute covering a colleague's lesson can log it. Giving a *mark* is stricter — see Walkthrough 4. And check 4 exists because the database cannot express it: keying `DiaryEntry` to `ClassSubjectTeacher` would make `(ClassId, SubjectId)` an identity in EF's change tracker, which breaks ending one assignment and creating its replacement in the same transaction.

Creating the register

```
var entry = mapper.Map<DiaryEntry>(dto);
entry.SchoolId = currentUser.RequireSchoolId();

await diaryEntries.AddAsync(entry);

foreach (var enrollment in await enrollments.GetActiveByClassAsync(dto.ClassId, ct))
{
    await attendances.AddAsync(new Attendance
    {
        DiaryEntryId = entry.Id,
        EnrollmentId = enrollment.Id,
        ClassId      = @class.Id,
        SchoolId     = entry.SchoolId,
        IsPresent    = true
    });
}

await diaryEntries.SaveChangesAsync(ct);
```

Logging the lesson opens the register in the same breath: one `Attendance` row per actively enrolled student, everyone marked present. The teacher then only corrects the exceptions, which is how a paper register works.

`entry.Id` is readable immediately after `AddAsync` because EF generates Guid keys client-side (Part 2, section 8). `Attendance` has no navigation property back to `DiaryEntry`, so EF cannot wire the id up for us — and it does not need to.

The whole thing — the lesson, thirty attendance rows, and thirty-one audit rows — is one `SaveChangesAsync` and therefore one transaction.

Walkthrough 4 — A teacher gives a mark

```
POST /api/marks
{ "enrollmentId": "...", "subjectId": "...", "gradingPeriodId": "...",
  "type": "Test", "value": 5, "date": "2026-03-04" }
```

Look at what the client does *not* send: no class, no teacher, no academic year, no school. A `Mark` row carries all of those, and the server derives every one.

```
var enrollment = await enrollments.GetByIdAsync(dto.EnrollmentId, ct)
                 ?? throw new NotFoundException("Enrollment", dto.EnrollmentId);

var @class = await classes.GetByIdAsync(enrollment.ClassId, ct)
             ?? throw new NotFoundException("Class", enrollment.ClassId);

var assignment = await assignments.GetActiveAsync(@class.Id, dto.SubjectId, ct)
                 ?? throw new BusinessException(
                     "No teacher is currently assigned to teach that subject to this class.");

if (currentUser.IsTeacher && assignment.TeacherId != currentUser.TeacherId)
    throw new ForbiddenException("Only the teacher assigned to this class-subject can give marks for
it.");
```

The chain is: enrollment → its class → the active teaching assignment for (that class, that subject). The caller cannot file a mark against a class, teacher or year that does not follow from the enrollment they named.

And note the first two lookups go through repositories, so the **school query filter** applies. An enrollment id from another school simply is not found — the tenancy check needs no code of its own here.

Stricter than a lesson

A lesson accepts any qualified teacher. A mark requires the *assigned* one. A substitute may record that a lesson happened; they may not grade the class. That difference is enforced twice — in the service, and by a five-column foreign key from `Mark` into `ClassSubjectTeacher` that forces class, subject, teacher and school to match the assignment row.

Calendar rules

```
if (gradingPeriod.AcademicYearId != @class.AcademicYearId)
    throw new BusinessException("That grading period belongs to a different academic year than this
class.");

if (dto.Date < gradingPeriod.StartDate || dto.Date > gradingPeriod.EndDate)
    throw new BusinessException($"The mark date must fall inside the grading period (...).");
```

The first is also enforced by the schema; checking it here turns a database constraint violation into a readable message. The second is application-only.

Building the row

```
var mark = new Mark
{
    EnrollmentId          = enrollment.Id,
    ClassId               = @class.Id,
    AcademicYearId        = @class.AcademicYearId,
    SchoolId              = currentUser.RequireSchoolId(),
    ClassSubjectTeacherId = assignment.Id,
    SubjectId             = assignment.SubjectId,
    TeacherId             = assignment.TeacherId,
    GradingPeriodId       = gradingPeriod.Id,
    Type = dto.Type, Value = dto.Value, Date = dto.Date
};
```

Every id comes from something already verified. Only `Type`, `Value` and `Date` come from the client, and the validator has already confirmed `Value` is 1–5 and the date is not in the future.

```
POST /api/marks → policy: AdminOrTeacher → ValidationEndpointFilter — value 1-5, date not future → PasswordChangeEndpointFilter → MarkService.CreateAsync → enrollment (school filter applies) → class (school filter applies) → active ClassSubjectTeacher → assigned teacher == caller? → grading period in the class's year? → date inside the period? → build Mark from verified ids → SaveChanges → SchoolScopeInterceptor — school matches? → AuditSaveChangesInterceptor → DB: 5-column FK re-checks the assignment ← 200 MarkListDto
```

What the four have in common

1. **Endpoints are thin.** Every one is a route, a policy, and a single service call.
2. **Services own the rules** and throw domain exceptions; they never build HTTP responses.
3. **Ids from the client are always re-fetched** through a repository, so the school filter decides whether they exist.
4. **`SchoolId` always comes from the token**, never the body.
5. **Important invariants are enforced twice** — once in a service for a readable error, once in the schema so no future code path can bypass it.